

# Tema 2 — Semáforos

---

Fuentes: Teórica 02-Semaforos, Práctica clase-02, Guía 2, Labo 1 (+ repaso en la práctica 3). **Ejercicio 2 del parcial.**

---

## 1. Qué es un semáforo

---

Es un **contador de permisos** (un entero  $\geq 0$ ) más un **conjunto de threads dormidos**. Tiene dos operaciones, las dos atómicas:

- `acquire()` (también llamada `wait` o `P`): si hay permisos, toma uno y sigue. Si no hay, **se duerme** (queda *blocked*, no gasta CPU).
- `release()` (también `signal` o `V`): si hay alguien dormido, despierta a uno. Si no, suma un permiso.

**En criollo:** una canasta de fichas en la puerta de un boliche. Para entrar agarrás una ficha. Si no hay fichas, te sentás a esperar. El que sale devuelve la ficha, o se la da directamente a uno de los que están esperando.

Invariantes (los usás para justificar):

- $S.V \geq 0$
- $S.V = \text{inicial} + \#release - \#acquire$  (contando solo los `acquire` completados)
- Mutex con  $S = \text{Semaphore}(1)$ :  $\#enSC + S.V = 1$ . De ahí sale que  $\#enSC \leq 1$  (exclusión mutua) y que no hay deadlock: si todos estuvieran bloqueados, sería  $S.V = 0$  y  $\#enSC = 0$ , y la suma daría  $0 \neq 1$ .

### Débiles vs fuertes (clave para los incisos de inanición)

- **Débil** (el default en el parcial y en Java: `new Semaphore(n)`): despierta a **cualquiera** de los que esperan. Encima, uno que recién llega se puede **colar** (*barging*). → Puede haber **inanición**.
- **Fuerte** (`new Semaphore(n, true)`): cola **FIFO**. Si hay N threads, a lo sumo N-1 te pasan adelante → **no hay inanición**.
- Mutex con semáforo débil y 3 o más threads: p y r se pueden turnar para siempre mientras q nunca entra.
- **Udding / Morris**: es posible hacer un mutex sin inanición usando solo semáforos débiles. Usa dos "puertas" y un único permiso: hasta que no se vacía la primera etapa no se arranca con la segunda (código en 3.10).

Regla práctica: **nunca** uses `availablePermits()` para decidir algo. Entre el `if` y el `acquire` otro thread te puede ganar (check-then-act). Si necesitás "intentar sin bloquearte", existe `tryAcquire()`.

---

## 2. Los "ladrillos" (patrones mínimos)

---

| Patrón                     | Código                                                                                            | En criollo                                                                                         |
|----------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Señal</b>               | <code>s = Sem(0); A: hacer A; s.release(); B: s.acquire(); hacer B</code>                         | "avisame cuando termines"                                                                          |
| <b>Rendezvous</b>          | <code>aLlego = Sem(0), bLlego = Sem(0); A: aLlego.release(); bLlego.acquire(); B: al revés</code> | "nos encontramos en la esquina y ninguno sigue hasta que llegue el otro" (labo: Andrea y Bernardo) |
| <b>Mutex</b>               | <code>m = Sem(1); m.acquire(); SC; m.release()</code>                                             | una sola llave                                                                                     |
| <b>Multiplex</b>           | <code>s = Sem(K)</code>                                                                           | "entran de a K": K fichas                                                                          |
| <b>Split / alternancia</b> | <code>puedeP = Sem(1), puedeQ = Sem(0); p: puedeP.acquire(); ...; puedeQ.release()</code>         | "ahora vos, ahora yo" (ping-pong)                                                                  |
| <b>Contador de eventos</b> | cada A hace release, cada F hace acquire                                                          | garantiza $\#F \leq \#A$                                                                           |

⚠ **Pedir varios permisos del mismo semáforo de a uno → deadlock.** Si hay 2 permisos y dos threads necesitan 2 cada uno, cada uno puede tomar 1 y quedarse esperando el segundo para siempre. Solución: tomarlos bajo un mutex, o con `acquire(k)`.

### 3. Problemas clásicos: código y cómo funciona

Notación: `Sem(n)` es `new Semaphore(n)` (débil) y `Sem(n, true)` es el fuerte. `acquire` / `release` es lo mismo que `wait` / `signal` de la teórica.

#### 3.1 Productor–Consumidor (buffer)

**En criollo:** una estantería con N lugares. Los productores ponen cosas y los consumidores sacan. No se puede sacar de una estantería vacía ni poner en una llena.

```
buffer[N]; inicio = 0; fin = 0
notEmpty = Sem(0)    // cuántas cosas hay para sacar
notFull  = Sem(N)    // cuántos lugares libres hay
mutexP = Sem(1); mutexC = Sem(1)    // un productor a la vez con "inicio", un consumidor a la vez con "fin"

productor:
    notFull.acquire()           // espero lugar libre
    mutexP.acquire(); buffer[inicio] = d; inicio = (inicio+1) % N; mutexP.release()
    notEmpty.release()         // aviso que hay algo

consumidor:
    notEmpty.acquire()          // espero que haya algo
```

```
mutexC.acquire(); d = buffer[fin]; fin = (fin+1) % N; mutexC.release()
notFull.release() // aviso que hay lugar
```

### Cómo funciona:

- `notFull` y `notEmpty` son contadores de eventos. El productor "gasta" un lugar libre y "produce" un elemento; el consumidor hace al revés.
- Son "semáforos partidos":  $\text{notEmpty.V} + \text{notFull.V} \leq N$ . Lo que falta para llegar a N son los elementos que alguien está poniendo o sacando en ese momento.
- `mutexP` protege `inicio` y `mutexC` protege `fin`. Como son índices distintos, un productor y un consumidor pueden trabajar a la vez. Sin `mutexC`, dos consumidores pueden leer el mismo `buffer[fin]` y saltarse otro.
- **Ojo con el orden:** primero `notFull` y después `mutexP`. En la versión clásica con **un solo** `mutex` compartido por productores y consumidores, al revés hay deadlock:
  - Con el buffer lleno, el productor toma `mutex` y se duerme en `notFull` **con el mutex tomado**.
  - El consumidor pasa `notEmpty` y se bloquea en `mutex`, así que nunca llega a hacer `notFull.release()` → espera circular.
  - Con `mutexP` / `mutexC` separados no hay deadlock (el consumidor no necesita `mutexP`), pero igual se usa este orden: **nunca dormir esperando una condición con un lock tomado**.

## 3.2 Filósofos comensales

**En criollo:** 5 filósofos en una mesa redonda con 5 tenedores, uno entre cada par. Para comer necesitás los dos tenedores de al lado. Si todos agarran el izquierdo al mismo tiempo, todos esperan el derecho para siempre: **deadlock** (espera circular).

### Intento ingenuo (tiene deadlock):

```
tenedores[5] = Sem(1) cada uno

filosofo(i):
    izq = i; der = (i+1) % 5
    while (true):
        pensar
        tenedores[izq].acquire()
        tenedores[der].acquire()
        comer
        tenedores[izq].release()
        tenedores[der].release()
```

Hay exclusión mutua sobre cada tenedor ( $\#Pi + \text{tenedores}[i].V = 1$ ), pero si los 5 ejecutan la primera línea a la vez, cada uno tiene su izquierdo y espera su derecho, que lo tiene el vecino. Nadie llega nunca a un `release`.

### (A) Reducir la concurrencia (sillas = N-1):

```
sillas = Sem(4)           // N-1; fuerte si piden sin inanición

filosofo(i):
    while (true):
        pensar
        sillas.acquire()   // a lo sumo 4 sentados
        tenedores[izq].acquire()
        tenedores[der].acquire()
        comer
        tenedores[izq].release()
        tenedores[der].release()
        sillas.release()
```

- **Sin deadlock (por absurdo):** si todos estuvieran bloqueados, hay a lo sumo 4 sentados y cada uno tiene 1 tenedor, o sea 4 tenedores tomados. Queda 1 libre en la mesa, y el filósofo que lo tiene a su derecha lo puede tomar. Absurdo.
- **Sin inanición (con `sillas` fuerte), por casos:**
  - Si  $i$  está trabado en su izquierdo, lo tiene el vecino  $i-1$  como derecho. Ese vecino ya tiene los dos, así que come y lo suelta.
  - Si está trabado en su derecho, el vecino  $i+1$  lo tiene y también estaría trabado en su derecho. Por inducción, todos trabados en su derecho, pero hay a lo sumo 4 sentados. Absurdo.
  - En `sillas` no espera para siempre: los sentados siempre terminan y el semáforo es FIFO.

### (B) Romper la simetría:

```
filosofo(i):
    if (i == 0) { izq = 1; der = 0 }           // el 0 toma primero el derecho
    else      { izq = i; der = (i+1) % 5 }
    ...mismo loop que el ingenuo...
```

Ahora todos toman primero el tenedor de **menor número**. Eso es un orden global, y con orden global no puede haber ciclo de espera. El filósofo 0 y el 4 compiten primero por el tenedor 0: el que pierde no tiene ningún tenedor en la mano, así que no traba a nadie.

**Lección general:** hay deadlock cuando hay **espera circular**. Se evita con orden total de adquisición, con límite de participantes, o tomando todo junto (con monitores, tema 3).

## 3.3 Lectores-Escritores ★ (el más importante)

**En criollo:** una biblioteca. Pueden leer muchos a la vez, pero el que escribe tiene que estar solo, sin lectores ni otros escritores.

a) **Prioridad lectores = patrón LIGHTSWITCH** (interruptor de luz):

```
cantLectores = 0; mutexL = Sem(1); escribir = Sem(1)    // escribir = "room vacío"
```

```
lector:
```

```
    mutexL.acquire()
    cantLectores++
    if (cantLectores == 1) escribir.acquire()    // el primero prende la luz
    mutexL.release()
    LEER
    mutexL.acquire()
    cantLectores--
    if (cantLectores == 0) escribir.release()    // el último la apaga
    mutexL.release()
```

```
escritor:
```

```
    escribir.acquire(); ESCRIBIR; escribir.release()
```

### Cómo funciona:

- `escribir` es el room. Un escritor lo toma para él solo. Los lectores lo toman **como grupo**: el primero que entra lo agarra y el último que sale lo suelta.
- `mutexL` hace atómicos el `cantLectores++ / --` y el `if`. Sin él, dos lectores podrían creer que los dos son "el primero" (los dos harían `escribir.acquire()` y el segundo se trabaría), o que ninguno es "el último".
- Si llega un lector mientras escribe un escritor, el primer lector se bloquea en `escribir` **con** `mutexL` **tomado**, y los demás lectores se bloquean en `mutexL`. Acá eso está bien: igual tenían que esperar al escritor, y el escritor no necesita `mutexL` para salir.
- **Problema:** si siempre llega algún lector antes de que se vacíe la pieza, `cantLectores` nunca vuelve a 0 y **el escritor espera para siempre** (inanición).

### b) Sin inanición = agregar un MOLINETE (turnstile):

```
molinete = Sem(1, true)    // FUERTE
```

```
lector:
```

```
    molinete.acquire(); molinete.release()    // pasar por el molinete
    mutexL.acquire(); cantLectores++; if (cantLectores == 1) escribir.acquire(); mutexL.release()
    LEER
    mutexL.acquire(); cantLectores--; if (cantLectores == 0) escribir.release(); mutexL.release()
```

```
escritor:
```

```
    molinete.acquire()                // me paro en el molinete
    escribir.acquire()
    ESCRIBIR
    molinete.release()
    escribir.release()
```

### Cómo funciona:

- Los lectores pasan por el molinete y lo sueltan enseguida. No los frena, salvo que alguien lo esté reteniendo.
- El escritor **lo toma y no lo suelta** mientras espera `escribir`. Los lectores nuevos se quedan en el molinete, los que ya estaban adentro terminan, el último suelta `escribir` y entra el escritor.
- Tiene que ser **fuerte**: si fuera débil, lectores recién llegados le podrían ganar el molinete al escritor para siempre (barging).

**c) Prioridad escritores:** si hay un escritor esperando, los lectores nuevos no entran.

```
cantLectores = 0; escritoresEsperando = 0
mutexL = Sem(1); mutexE = Sem(1); mutexP = Sem(1)
leer = Sem(1); escribir = Sem(1)

lector:
    mutexP.acquire()           // a lo sumo UN lector compite por "leer"
    leer.acquire()             // puerta que cierran los escritores
    mutexL.acquire()
    cantLectores++
    if (cantLectores == 1) escribir.acquire()
    mutexL.release()
    leer.release()
    mutexP.release()
    LEER
    mutexL.acquire()
    cantLectores--
    if (cantLectores == 0) escribir.release()
    mutexL.release()

escritor:
    mutexE.acquire()
    escritoresEsperando++
    if (escritoresEsperando == 1) leer.acquire()    // el primer escritor cierra la puerta
    mutexE.release()
    escribir.acquire()
    ESCRIBIR
    escribir.release()
    mutexE.acquire()
    escritoresEsperando--
    if (escritoresEsperando == 0) leer.release()    // el último escritor la abre
    mutexE.release()
```

### Cómo funciona, semáforo por semáforo:

| Semáforo | Qué hace                                              | Quién lo usa                                   |
|----------|-------------------------------------------------------|------------------------------------------------|
| escribir | el room: exclusión sobre el recurso                   | escritores (de a uno) y lectores (como grupo)  |
| mutexL   | protege cantLectores                                  | lectores                                       |
| leer     | la puerta de entrada de los lectores                  | escritores (como grupo) y lectores (de pasada) |
| mutexE   | protege escritoresEsperando                           | escritores                                     |
| mutexP   | que haya <b>a lo sumo un lector</b> esperando en leer | lectores                                       |

- escritoresEsperando cuenta a los que esperan **y** al que escribe (se decrementa después de escribir).
- Hay **dos lightswitches**: el de lectores sobre escribir y el de escritores sobre leer. Mientras haya algún escritor, leer queda cerrado y no entra ningún lector nuevo. Los que ya estaban terminan, el último suelta escribir, y los escritores pasan de a uno, varios seguidos.
- El lector toma leer y mutexP **solo para anotarse** y los suelta antes de LEER, así que pueden leer muchos a la vez.
- mutexP es el truco: sin él, podría haber muchos lectores encolados en leer, y con un semáforo débil el escritor podría perder la carrera muchas veces. Con mutexP, el escritor compite contra un solo lector.
- **Contra**: ahora los **lectores** pueden sufrir inanición si nunca deja de haber escritores.

### 3.4 Puente de una vía / Saldungaray (labo 1, apunte clase 3, Guía 2 ej 12)

**En criollo**: es lectores-escritores **simétrico**. Autos del mismo sentido pueden ir juntos; del sentido contrario, no.

#### Parte 1: CaminoUnaVia (labo 1)

```

A = 0; B = 1
turnstile = Sem(1, true)           // el labo lo tiene débil; para justificar sin inanición,
FUERTE
resource = Sem(1)                  // el puente: lo tiene un sentido a la vez
countMutex[2] = { Sem(1), Sem(1) } // uno por sentido
count[2] = { 0, 0 }                // autos de cada sentido en el puente

entrar(dir):
    turnstile.acquire()
    countMutex[dir].acquire()
    count[dir]++
    if (count[dir] == 1) resource.acquire() // el primero de mi sentido toma el puente
    countMutex[dir].release()
    turnstile.release()

salir(dir):
    countMutex[dir].acquire()

```

```

count[dir]--
if (count[dir] == 0) resource.release() // el último de mi sentido lo suelta
countMutex[dir].release()

auto:
while (true):
    dir = sentido al azar
    entrar(dir); cruzarPuede(); salir(dir)

```

### Cómo funciona:

- `count[dir]` + `countMutex[dir]` + `resource` forman **un lightswitch por sentido**, los dos sobre el mismo `resource`. Mientras haya un auto de A cruzando, `count[A] > 0` y `resource` sigue tomado por A, así que ningún auto de B puede entrar. Eso da la **exclusión entre sentidos**, y varios del mismo sentido cruzan juntos.
- `turnstile` se retiene **durante todo** `entrar`, inclusive mientras el primer auto de B está bloqueado en `resource.acquire()` esperando que A se vacíe. Es a propósito:
  - Mientras B1 lo retiene, nadie más (de ningún sentido) pasa el molinete.
  - Los autos de A que ya estaban terminan de cruzar, el último suelta `resource` y B1 entra.
  - Sin molinete, A podría seguir entrando para siempre: es el problema de prioridad lectores, pero de los dos lados.
- **Sin deadlock:** el único que se duerme con algo tomado es el primero de un sentido, que tiene `turnstile` y `countMutex[dir]` y espera `resource`. Pero `resource` lo tiene el otro sentido, y para soltarlo esos autos solo necesitan su propio `countMutex`, que está libre.

¿Por qué el molinete tiene que ser fuerte? Traza con `turnstile` débil:

1. El sentido A tiene tránsito continuo: `count[A]` nunca llega a 0.
2. Llega B1 y queda esperando en `turnstile.acquire()`, porque otro auto se está registrando.
3. Se libera `turnstile` y justo llega A6. Como el semáforo es débil, A6 **se cuela** (barging), se registra y lo suelta.
4. Llega A7 y se repite el paso 3, y así para siempre. B1 nunca pasa el molinete.

Con `Sem(1, true)` nadie que recién llega le gana a alguien que ya estaba en la cola. B1 espera a lo sumo a los que tenía adelante, y cada uno retiene el molinete un tiempo acotado. El apunte avisa: *"Tenerlo presente si piden justificar no starvation con precisión (Ejem, un parcial, ejem)"*.

**"Máximo K en el puente" (Guía 2 ej 12b):** se agrega un multiplex.

```

lugares = Sem(K) // K = 3 en la guía

auto:
    entrar(dir) // turnstile + lightswitch, igual que antes
    lugares.acquire() // hay a lo sumo K sobre el puente
    cruzarPuede()

```



```
lugares.release()
salir(dir)
```

El multiplex va **después** del lightswitch: el auto ya "reservó" el sentido y espera lugar. Solo esperan lugar autos del mismo sentido, así que no hay deadlock. Para el 12c: con `lugares` o `turnstile` débiles puede haber inanición; con los dos fuertes, no.

**Parte 2: CaminoUnaViaFIFO**, "no se pasan entre sí" (un auto que entró después no puede salir antes). `turnstile` y `resource` solo deciden **qué sentido** cruza. Para ordenar las salidas hace falta **paso de testigo**: cada auto tiene su propio semáforo.

```
queueMutex = Sem(1)
exitQueue = cola de semáforos // uno por auto en el puente, en orden de llegada

entrar(dir):
    ...turnstile + lightswitch, igual que en la Parte 1...
    queueMutex.acquire()
    miTurno = Sem(0) // mi semáforo privado, arranca cerrado
    esElPrimero = exitQueue.isEmpty()
    exitQueue.addLast(miTurno)
    queueMutex.release()
    if (esElPrimero) miTurno.release() // si no hay nadie adelante, me abro solo

salir(dir):
    miTurno.acquire() // espero a que me toque salir
    ...apagar el lightswitch, igual que en la Parte 1...
    queueMutex.acquire()
    exitQueue.removeFirst() // me saco (soy el primero)
    siguiente = exitQueue.peekFirst()
    queueMutex.release()
    if (siguiente != null) siguiente.release() // le paso el testigo al que entró después de mí
```

### Cómo funciona:

- **Invariante:** entre los autos del puente, a lo sumo un `miTurno` está abierto, y es el del más antiguo.
  - Vale al principio: el que encuentra la cola vacía abre el suyo.
  - `entrar` lo mantiene: el nuevo se agrega al final, cerrado.
  - `salir` lo mantiene: el que sale tenía el abierto y abre el del siguiente.
- Entonces el único que puede completar `salir` es el más antiguo: **salen en orden de llegada**, sin que ningún semáforo compartido sea fuerte, porque nadie espera en un semáforo compartido.
- La versión del labo, con capacidad `BRIDGE_CAPACITY`, es igual, pero usa un arreglo circular de semáforos (`head`, `nextSpot`, `queued`) en vez de la cola, más `spots = Sem(K)`.

## 3.5 Barrera reutilizable (Volcán Lanín, labo 1)

**En criollo:** nadie arranca el siguiente tramo hasta que llegan todos a la pirca.

```
n = cantidad de threads; count = 0
mutex = Sem(1); turnstile = Sem(0); turnstile2 = Sem(0)

esperar():
    mutex.acquire()
    count++
    if (count == n) turnstile.release(n)    // el último en llegar abre para los n
    mutex.release()
    turnstile.acquire()                    // fase 1: espero que lleguen todos

    mutex.acquire()
    count--
    if (count == 0) turnstile2.release(n)   // el último en salir abre la fase 2
    mutex.release()
    turnstile2.acquire()                   // fase 2: espero que salgan todos
```

### Cómo funciona:

- **Fase 1:** cada thread se anota en `count` y se duerme en `turnstile`, que arranca en 0. El n-ésimo da **n permisos** de una, uno para cada thread, incluido él mismo.
- **Fase 2:** cada uno se desanota, y el último en hacerlo (`count == 0`) da n permisos en `turnstile2`.
- **¿Por qué dos molinetes?** Con uno solo, un thread rápido podría salir, dar la vuelta al loop, volver a entrar a `esperar()` y agarrar un permiso de la ronda anterior que era para otro, o dejar mal `count`. El segundo molinete garantiza que nadie empieza la ronda siguiente hasta que **todos** salieron de la actual.
- Al final de cada ronda los dos semáforos y `count` vuelven a 0, así que la barrera se puede reusar.

## 3.6 Fumadores (Patil)

**En criollo:** para armar un cigarrillo hacen falta tabaco, papel y fósforos. Cada fumador tiene infinito de **una** cosa. El agente pone **dos** cosas sobre la mesa. Tiene que agarrarlas el fumador al que le faltan justo esas dos.

### Los agentes (no se pueden modificar):

```
tabaco = Sem(0); papel = Sem(0); fosforos = Sem(0)
activarAgente = Sem(1)

agenteTP:                                // hay 3: TP, PF, TF
    while (true):
        activarAgente.acquire()
```

```
tabaco.release()
papel.release()
```

### Solución ingenua (deadlock):

```
fumadorT:                                // tiene tabaco
while (true):
    papel.acquire()
    fosforos.acquire()
    armar y fumar
    activarAgente.release()
```

Traza: el agente TP pone tabaco y papel. `fumadorT` toma el papel y `fumadorF` (el que tiene fósforos) toma el tabaco. Ahora `fumadorT` espera fósforos y `fumadorF` espera papel, que ya no hay, y nadie hace `activarAgente.release()` → deadlock.

**Solución con gestores:** un thread gestor por ingrediente, que decide quién fuma.

```
hayTabaco = hayPapel = hayFosforo = false
mutexGestores = Sem(1)
fumadorTabaco = Sem(0); fumadorPapel = Sem(0); fumadorFosforo = Sem(0)

gestorT:                                // hay 3: gestorT, gestorP, gestorF
while (true):
    tabaco.acquire()
    mutexGestores.acquire()
    if (hayPapel)        fumadorFosforo.release()    // tabaco + papel → le toca al que tiene
fósforos
    else if (hayFosforo) fumadorPapel.release()      // tabaco + fósforo → al que tiene papel
    else                 hayTabaco = true           // soy el primero: lo dejo anotado
    mutexGestores.release()

fumadorT:                                // tiene tabaco
while (true):
    fumadorTabaco.acquire()
    hayPapel = false; hayFosforo = false            // "tomo" papel y fósforos
    armar y fumar
    activarAgente.release()
```

### Cómo funciona:

- De los dos ingredientes que pone el agente, el primer gestor que despierta no encuentra nada y **anota** el suyo. El segundo encuentra el del primero y **despierta al fumador correcto**.
- `mutexGestores` hace que las dos decisiones no se pisen. Sin él, los dos gestores podrían ver `false`, anotar los dos, y no despertaría nadie.

- El fumador limpia los booleanos sin mutex. Es seguro porque el agente está bloqueado en `activarAgente` hasta que el fumador lo libere, así que no hay ningún gestor tocándolos.

**Lección:** cuando la condición es **compuesta** ("A y B a la vez"), los acquire sueltos no alcanzan. Hace falta una variable de estado bajo un mutex (o un monitor, tema 3: el "Mostrador").

### 3.7 Barbero dormilón (Dijkstra)

**En criollo:** una peluquería con un barbero y n sillas de espera. Si no hay nadie, el barbero duerme. El cliente que llega y encuentra todo lleno se va.

```
genteEnBarberia = 0 // cuenta hasta n+1 (los n sentados + el que se corta)
mutexClientes = Sem(1)
solicitudCorte = Sem(0) // cliente → barbero: "quiero cortarme"
listoParaCortar = Sem(0, true) // barbero → cliente: "pasá" (FUERTE para respetar el orden)
corteTerminado = Sem(0) // barbero → cliente: "terminé"
clienteRetirado = Sem(0) // cliente → barbero: "ya me fui"

cliente:
    meVoy = false
    mutexClientes.acquire()
    if (genteEnBarberia == n+1) meVoy = true
    else genteEnBarberia++
    mutexClientes.release()
    if (!meVoy):
        solicitudCorte.release()
        listoParaCortar.acquire()
        cortarseElPelo()
        corteTerminado.acquire()
        mutexClientes.acquire(); genteEnBarberia--; mutexClientes.release()
        clienteRetirado.release()

barbero:
    while (true):
        solicitudCorte.acquire() // duermo si no hay nadie
        listoParaCortar.release()
        cortarPelo()
        corteTerminado.release()
        clienteRetirado.acquire()
```

#### Cómo funciona:

- **"Si está lleno, me voy"** se decide con el contador bajo `mutexClientes`, no con `availablePermits` (check-then-act). Cuenta hasta n+1 para no tener que separar el caso "alguien se está cortando y las sillas están vacías".

- **Inicio del corte:** `solicitudCorte` / `listoParaCortar` es un rendezvous. El cliente avisa que llegó y espera que el barbero lo llame; el barbero duerme hasta que haya un pedido y llama a uno.
- **Fin del corte:** `corteTerminado` / `clienteRetirado` es otro rendezvous. El cliente no se va antes de que termine el corte, y el barbero no llama al siguiente hasta que el anterior se descontó y se fue.
- `cortarseElPelo()` y `cortarPelo()` corren a la vez porque están entre los dos rendezvous.
- `listoParaCortar` es **fuerte** para atender en orden de llegada. Sin semáforos fuertes: cada cliente pone *su propio* semáforo en un buffer (un productor-consumidor de semáforos) y el barbero los va liberando en orden FIFO (paso de testigo, 3.8).

### 3.8 Paso de testigo (baton passing) = FIFO sin semáforos fuertes

**En criollo:** en vez de que todos esperen en la misma puerta (donde el portero deja pasar a cualquiera), cada uno tiene **su propio timbre**. Cuando te toca, te lo tocan a vos.

La versión "objetosa" del apunte de la clase 3 (es la misma cola de la Parte 2 del puente):

```
FilaDeSalida:
    mutex = Sem(1); fila = cola de semáforos

    anotarse():
        mutex.acquire()
        miTurno = Sem(0)
        esElPrimero = fila.isEmpty()
        fila.addLast(miTurno)
        mutex.release()
        if (esElPrimero) miTurno.release()
        return miTurno           // el thread después hace miTurno.acquire()

    avisarAlSiguiente():
        mutex.acquire()
        fila.removeFirst()
        proximo = fila.peekFirst()
        mutex.release()
        if (proximo != null) proximo.release()
```

- **Invariante:** entre los que están en la fila, a lo sumo un semáforo está abierto, y es el del más antiguo. Por eso pasan en orden de llegada.
- El `release` del siguiente se hace **afuera** del mutex. No hace falta que sea atómico con la cola: el semáforo guarda el permiso aunque su dueño todavía no haya llegado al `acquire`.
- Este patrón generalizado es un **monitor hecho a mano** (tema 3).

### 3.9 Venta de tickets (labo 1)

**En criollo:** muchos pueden mirar cuántos tickets quedan, pero comprar es de a uno. Y comprás solo si lo que miraste sigue siendo cierto.

```
available[tipo]; version = 0
readers = 0; readersMutex = Sem(1)
turn = Sem(1, true)                // el room, fuerte

ver(tipo):                          // lector: lightswitch sobre turn
    readersMutex.acquire(); readers++; if (readers == 1) turn.acquire(); readersMutex.release()
    lectura = (available[tipo], version)
    readersMutex.acquire(); readers--; if (readers == 0) turn.release(); readersMutex.release()
    return lectura

comprar(tipo, versionLeida):        // escritor: exclusivo
    turn.acquire()
    if (version != versionLeida) ok = false           // alguien compró mientras tanto
    else if (available[tipo] == 0) ok = false
    else { available[tipo]--; version++; ok = true }
    turn.release()
    return ok

intentarComprar(tipo):
    while (true):
        l = ver(tipo)
        if (l.cantidad == 0) return false
        if (comprar(tipo, l.version)) return true
        // cambió la versión: releo y reintento
```

### Cómo funciona:

- `ver` y `comprar` son lectores-escritores (3.3a) sobre `turn`.
- Entre el `ver` y el `comprar` no hay ningún lock, así que otro comprador se puede meter. El **número de versión** lo detecta: `comprar` solo descuenta si nadie compró desde tu lectura. Si cambió, releés y reintentás. Es la idea "optimista" que vuelve en el tema 4.
- `turn` es fuerte para que los compradores no esperen para siempre detrás de un flujo de lectores.

### 3.10 Mutex sin inanición con semáforos débiles (Udding)

Dijkstra conjeturó que no se podía; Morris (1979) y Udding (1986) mostraron que sí. Viene de la práctica 2; no aparece en la guía ni en el simulacro, así que alcanza con entender la idea.

```
threadsP1 = 0; threadsP2 = 0
puerta1 = Sem(1); puerta2 = Sem(0); mutex = Sem(1)

thread p:
```

```

while (true):
    puerta1.acquire(); threadsP1++; puerta1.release()    // me anoto en la etapa 1

    mutex.acquire()
    puerta1.acquire()
    threadsP1--; threadsP2++                               // paso de la etapa 1 a la 2
    if (threadsP1 > 0) puerta1.release()                   // quedan en la etapa 1: que pase otro
    else puerta2.release()                                 // se vació la etapa 1: abro la 2
    mutex.release()

    puerta2.acquire()
    threadsP2--
    SC
    if (threadsP2 > 0) puerta2.release()                   // quedan en la etapa 2: que entre otro
    else puerta1.release()                                 // se vació: vuelvo a abrir la etapa 1

```

### Cómo funciona:

- Hay **un solo permiso**, que está en `puerta1`, en `puerta2` o lo tiene un thread. Por eso hay exclusión mutua.
- Los threads entran por **tandas**. Mientras la tanda de la etapa 2 hace la SC, `puerta1` está cerrada, así que nadie nuevo se anota. La tanda es finita y cada uno pasa una vez; recién cuando se vacía se vuelve a abrir la etapa 1.
- `mutex` hace que a lo sumo un thread espere en el segundo `puerta1.acquire()`. Así, el que quiere anotarse (el primer `puerta1.acquire()`) no queda siempre perdiendo contra los que pasan de etapa.

## 4. Checklist para justificar una solución con semáforos

1. Para qué es cada semáforo, con qué valor arranca, y **qué invariante** mantiene.
2. **Sin race conditions:** cada variable compartida se toca con su mutex tomado.
3. **Sin deadlock:**
  - no hay espera circular (orden global);
  - nadie se duerme con un mutex tomado que necesita quien lo va a despertar;
  - todo `acquire` tiene su `release` en todos los caminos.
4. **Inanición:** con semáforos débiles puede haber sobrepasos sin cota (mostrá la traza). Con fuertes, la cota es la cantidad de threads que ya estaban adelante en la cola.
5. Suposiciones explícitas: "asumo semáforos débiles", "el turnstile es fuerte porque...".

## 5. Java (práctica 2 / labo)

- **Crear threads:** `new Thread(() -> ...).start()`.
  - `.run()` **no** crea un thread nuevo, ejecuta en el mismo.

- `join()` espera a que termine. Primero lanzá todos y después hacé todos los joins.
- **`interrupt()`** : pide parar, de forma cooperativa. No te comas la `InterruptedException` : relanzala o volvé a prender el flag.
- **`volatile`** : visibilidad, no atomicidad.
- **`ReentrantLock`** :
  - `lock()` y `unlock()` **en un `finally`** .
  - Tiene **dueño**: solo lo libera el que lo tomó.
  - Es reentrante (el mismo thread lo puede volver a tomar).
  - `new ReentrantLock(true)` es fair.
- **`Semaphore`** : no tiene dueño (cualquiera puede hacer release) y no es reentrante. Métodos: `acquire(k)` , `tryAcquire()` , `new Semaphore(n, true)` para el fuerte.
- **Por dentro** (cultura general): AQS (un entero atómico con CAS + una cola) → `LockSupport.park()` → futex del sistema operativo.

## 6. Guía 2 — qué patrón entrena cada ejercicio

| Ej  | Patrón                                                                                            |
|-----|---------------------------------------------------------------------------------------------------|
| 1–3 | señales / precedencia (ya los resolviste)                                                         |
| 4   | contadores de eventos ( $\#F \leq \#A$ )                                                          |
| 5   | split semaphores (alternancia, diferencia $\leq 1$ , ABB)                                         |
| 6   | señalización generador ↔ acumulador (en Java)                                                     |
| 7   | gimnasio: recursos múltiples (aparato + k discos): orden de adquisición / tomar k juntos          |
| 8   | bolitas: capacidad que cambia con los participantes + consumidores que toman pares (¡deadlock!)   |
| 9   | bote: barrera / "esperar N a bordo" + dos costas                                                  |
| 10  | planta: señales entre vehículo y máquina (rendezvous)                                             |
| 11  | <b>baño + limpieza: lightswitch (a), turnstile / prioridad (b), dónde va el lightswitch (c) ★</b> |
| 12  | <b>puede: lightswitch por sentido (a), + multiplex (b), inanición (c) ★</b>                       |